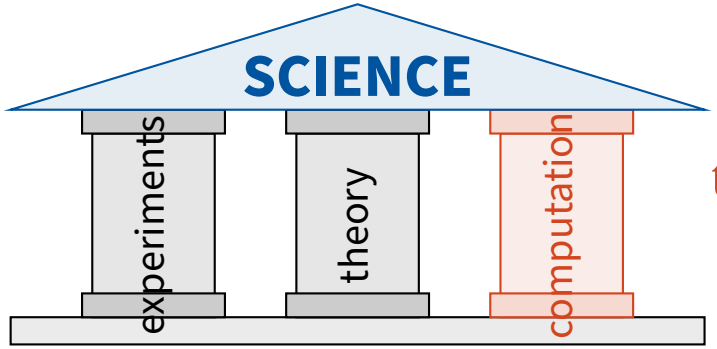
Chapter 0 — Motivation

Why this unit?

MTH3340 · Numerical Methods for PDEs

1 The three pillars of science

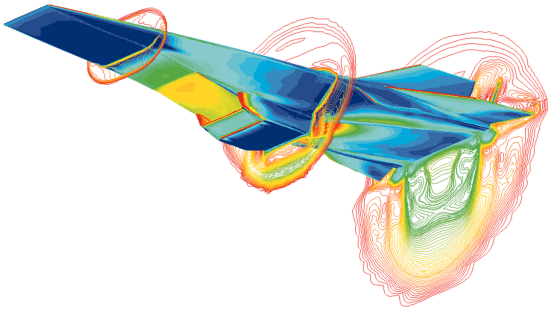


the third pillar,
~ 50 years old

- Observe and measure; build *theories* (models) that explain and predict. Experiments confirm or refute theories. E.g., the Higgs boson; predicted in the 1970s, detected in 2012.
- Computational science uses computers to *approximate* complex scientific problems. Hardware improved enormously, but algorithmic improvements have mattered even more.

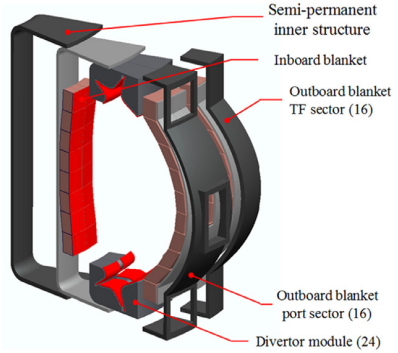
Predictive science

Simulate what is too expensive, too complex, too dangerous, or impossible to observe in real life.



virtual wind tunnels

CFD of the X-43A at Mach 7 (NASA)

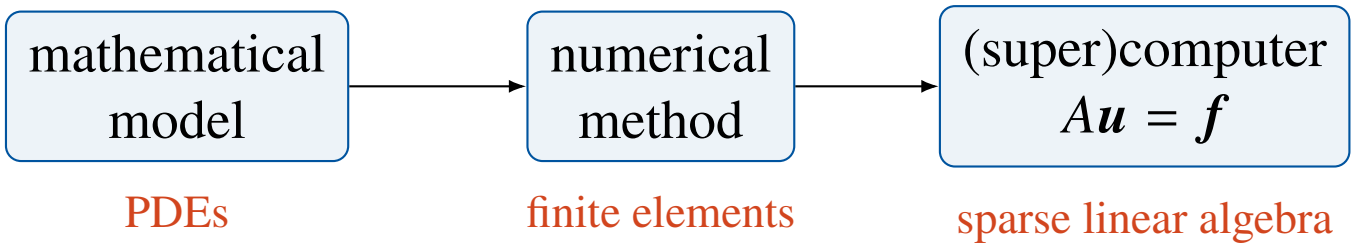


fusion reactor breeding blanket

K-DEMO design; no facility exists yet (ITER)

The synergy between experimental, theoretical and computational science is essential. In engineering, this area is called computational science and engineering (CSE). Applications in medicine, Earth sciences, biology, aeronautics, finance, . . .

The ingredients of CSE



A multidisciplinary field; mathematics, computer science, and the application sciences (physics, chemistry, biology, . . .).

MTH3340 is about a specific class of models, partial differential equations (PDEs), and a specific class of numerical methods, finite element methods.

2 PDEs are everywhere

Maxwell (electromagnetism), Navier–Stokes (fluids), elasticity (solids), Schrödinger (quantum mechanics), cell migration in living tissues, . . .

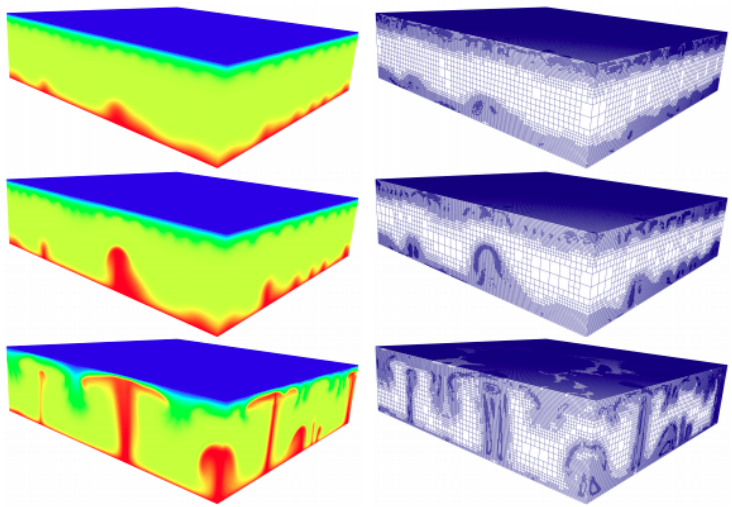
E.g., mantle convection in the Earth:

$$\begin{cases} -\boldsymbol{\nabla} \cdot (2\mu\, \boldsymbol{\varepsilon}(\boldsymbol{u})) + \boldsymbol{\nabla} p = \rho(T)\, \boldsymbol{g} & \text{(momentum)} \\ \boldsymbol{\nabla} \cdot \boldsymbol{u} = 0 & \text{(mass)} \\ \partial_t T + \boldsymbol{u} \cdot \boldsymbol{\nabla} T - \boldsymbol{\nabla} \cdot (\kappa \boldsymbol{\nabla} T) = H & \text{(energy)} \end{cases}$$

with unknowns the velocity $\boldsymbol{u}$, the pressure p and the temperature T of the mantle.

Impossible to solve analytically (unless very simple PDEs on very simple domains). Computers cannot solve PDEs either; they only add, subtract, multiply, divide and compare. We must *approximate*.

From functions to numbers: meshes



mantle convection; temperature (left) and the mesh that adapts to it (right)

- Numerical methods for PDEs rely on meshes (e.g., triangulations).
- The unknowns are no longer functions but values at the nodes; our unknown is an array of real numbers.
- Derivatives are replaced by simple algebraic relations between neighbouring values.

We commit an error, but we can prove how large it is and how it goes to zero as the mesh is refined. This is *numerical analysis*, and it is what this unit teaches.

Which method? Why finite elements?

You probably know the finite difference method; approximate derivatives with Taylor expansions on a grid. Hard to apply to complex geometries!

More advanced methods; finite volumes (favoured for hyperbolic PDEs) and finite elements (applicable to any PDE). In this unit, finite elements.

The finite element idea

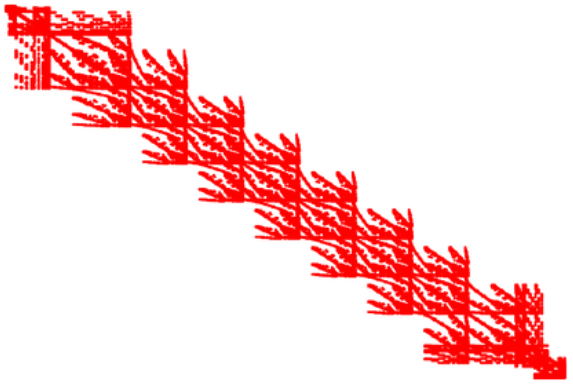
The solution $u(x)$ belongs to a known ∞-dimensional function space V. Replace V by a finite-dimensional space V_h that is *close enough* to V,

$$u \in V \quad \longrightarrow \quad u_h \in V_h, \qquad \dim V_h < \infty.$$

It relies on *weak forms* of PDEs and functional analysis; a brief introduction to both is part of this unit (starting next lecture!).

The end of the pipeline: linear systems

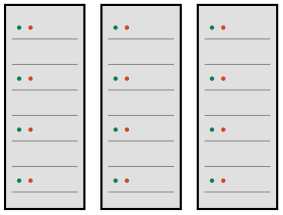
- After discretisation, the PDE becomes a (set of) linear systems $A\boldsymbol{u} = \boldsymbol{f}$.
- A computer *can* solve those; e.g., Gaussian elimination only involves basic operations.
- The matrices are very sparse; most entries are zero, with nice patterns.



sparsity pattern of a PDE discretisation; dots are the non-zeros

The number of unknowns in complex 3D problems can be HUGE; the largest PDE simulations so far involve $\sim 10^{13}$ equations.

Supercomputers, and why algorithms still win



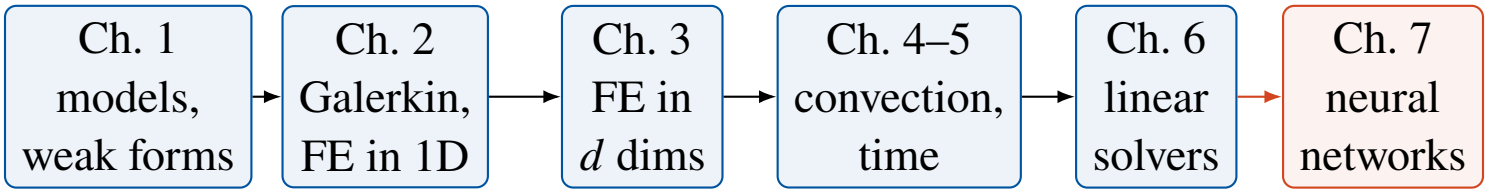
- The largest machines have millions of cores working together (see the TOP500 list; Australia's Gadi is on it).
- Exploiting them efficiently is high-performance scientific computing.

Algorithms have had even more impact than hardware. E.g., meshes that *adapt* to the solution, with fine resolution only where needed, reduce the cost of mantle convection simulations by a factor of ~ 1000 compared to uniform brute force. We will meet this idea again (adaptivity, chapter 7).

Software; and this unit

PDE software libraries are complex, easily millions of lines of code, and many are open source; you can read, download, use and modify them (e.g., on GitHub).

In this unit we use `Gridap`, a finite element library in the `Julia` language, developed by the course author and co-workers.



Theory and computation, together; the mathematics of finite elements, plus hands-on tutorials solving real PDEs. Next lecture, we start with the mathematical models.